

Teoria Współbieżności

Laboratorium 5

Problem pięciu filozofów

Kacper Bieniasz

4 listopada 2024

1 Dane techniczne sprzętu

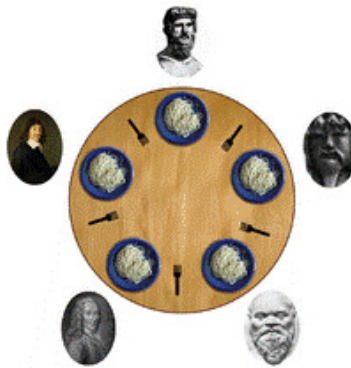
Obliczenia zostały wykonane na komputerze o następującej specyfikacji:

- Procesor: AMD Ryzen 7 5800U
- Pamięć RAM: 16 GB DDR4 3200 MHz (2×8GB)
- System operacyjny: Windows 11 Home x64

2 Treść zadań

Celem ćwiczenia jest zapoznanie z jednym z klasycznych problemów teorii współbieżności - problemem pięciu filozofów (autor E.W. Dijkstra).

2.1 Problem pięciu filozofów



Rysunek 1: Problem pięciu filozofów

- Każdy filozof zajmuje się głównie myśleniem.
- Od czasu do czasu potrzebuje zjeść.
- Do jedzenia potrzebne mu są oba widelce po jego prawej i lewej stronie.
- Jedzenie trwa skończoną (ale nieokreślona z góry) ilość czasu, po czym filozof widelce odkłada i wraca do myślenia.
- Cykl powtarza się od początku.

2.2 Treść zadania

W celu rozwiązania problemu pięciu filozofów wykorzystać dostarczony szkielet programu oraz zrealizować go na poniższe sposoby:

- Zaimplementować trywialne rozwiązanie z symetrycznymi filozofami. Zaobserwować problem blokady.
- Zaimplementować rozwiązanie z widelcami podnoszonymi jednocześnie. Jaki problem może tutaj wystąpić?
- Zaimplementować rozwiązanie z lokajem.

Wykonać pomiary dla każdego rozwiązania i wywnioskować co ma wpływ na wydajność każdego rozwiązania. Dodatkowo zaproponować autorskie rozwiązanie, inne niż powyższe.

3 Implementacja zadania

Dołączony szkielet programu przedstawia kod poniższy kod (1).

```
1 class Widelec {
2     public void podnies() {
3     }
4     public void odloz() {
5     }
6 }
7 class Filozof extends Thread {
8     private int _licznik = 0;
9     public void run() {
10         while (true) {
11             // jedzenie
12             ++_licznik;
13             if (_licznik % 100 == 0) {
14                 System.out.println("Filozof: " + Thread.currentThread() +
15                     "jadłem " + _licznik + " razy");
16             }
17             // koniec jedzenia
18         }
19     }
20 }
21 public class Fil5mon {
22     public static void main(String[] args) {
23     }
24 }
```

Kod Java 1: Szkielet programu

3.1 Implementacja rozwiązania trywialnego

Kod (2) zawiera implementację rozwiązania trywialnego polegającego na tym, że każdy z filozofów najpierw próbuje podnieść widelec ze swojej lewej strony. Bardzo łatwo możemy dostrzec jaki będzie efekt takiego rozwiązania. Jeżeli każdy filozof będzie czekał z jednym widelcem to, żaden z nich nigdy nic nie zje. Klasa **Widelec** w implementacji i działaniu przypomina semafor. Klasa **Filozof** realizuje mechanizm podniesienia oraz odłożenia widelców. Natomiast klasa **Fil5mon** tworzy widelce, filozofów oraz rozpoczyna pracę każdego wątku.

```
1 class Widelec {
2     private boolean zajety = false;
3     public synchronized void podnies() {
4         while (zajety) {
5             try {
```

```

6         wait();
7     } catch (InterruptedException e) {
8         Thread.currentThread().interrupt();
9     }
10 }
11 zajety = true;
12 }
13 public synchronized void odloz() {
14     zajety = false;
15     notify();
16 }
17 }
18 class Filozof extends Thread {
19     private final Widelec lewyWidelec;
20     private final Widelec prawyWidelec;
21     private int _licznik = 0;
22     public Filozof(Widelec lewyWidelec, Widelec prawyWidelec) {
23         this.lewyWidelec = lewyWidelec;
24         this.prawyWidelec = prawyWidelec;
25     }
26     public void run() {
27         while (true) {
28             lewyWidelec.podnies();
29             prawyWidelec.podnies();
30             ++_licznik;
31             try {
32                 Thread.sleep((int)(50*Math.random()));
33             } catch (InterruptedException e) {
34                 throw new RuntimeException(e);
35             }
36             if (_licznik % 100 == 0) {
37                 System.out.println("Filozof: " + Thread.currentThread().getName() +
38                     " jadlem " + _licznik + " razy");
39             }
40             prawyWidelec.odloz();
41             lewyWidelec.odloz();
42         }
43     }
44 }
45 public class Fil5mon {
46     public static void main(String[] args) {
47         int liczbaFilozofow = 5;
48         Widelec[] widelce = new Widelec[liczbaFilozofow];
49         Filozof[] filozofowie = new Filozof[liczbaFilozofow];
50         for (int i = 0; i < liczbaFilozofow; i++) {
51             widelce[i] = new Widelec();
52         }
53         for (int i = 0; i < liczbaFilozofow; i++) {
54             Widelec lewyWidelec = widelce[i];
55             Widelec prawyWidelec = widelce[(i + 1) % liczbaFilozofow];
56             filozofowie[i] = new Filozof(lewyWidelec, prawyWidelec);
57             filozofowie[i].start();
58         }
59     }
60 }

```

Kod Java 2: Implementacja rozwiązania trywialnego

Przykładowe uruchomienie programu spowodowało szybkie zaistnienie problemu blokady. Log z konsoli wygląda następująco:

Filozof: Thread-2 jadlem 100 razy

```

Filozof: Thread-2 jadlem 200 razy
Filozof: Thread-2 jadlem 300 razy
Filozof: Thread-2 jadlem 400 razy
Filozof: Thread-2 jadlem 500 razy
Filozof: Thread-2 jadlem 600 razy
Filozof: Thread-2 jadlem 700 razy
Filozof: Thread-2 jadlem 800 razy
Filozof: Thread-2 jadlem 900 razy
Filozof: Thread-2 jadlem 1000 razy
Filozof: Thread-0 jadlem 100 razy
Filozof: Thread-0 jadlem 200 razy
Filozof: Thread-0 jadlem 300 razy
Filozof: Thread-0 jadlem 400 razy
Filozof: Thread-2 jadlem 1100 razy
Filozof: Thread-2 jadlem 1200 razy
Filozof: Thread-2 jadlem 1300 razy
Filozof: Thread-0 jadlem 500 razy

```

Jak możemy zauważyć tylko dwóch filozofów był w stanie jeść. To zachowanie programu jest przewidywalne ponieważ, żaden z nich nie współdzieli tego samego widelca. Dodanie opóźnienia symulującego jedzenie powoduje wcześniejsze zaistnienie problemu blokady. Dodatkowo zamiana odkładana widelców, tak, że najpierw również odkładamy lewy, też prowadzi do blokady, ale z innym rezultatem, log z konsoli dla tej sytuacji wygląda następująco:

```

Filozof: Thread-0 jadlem 100 razy
Filozof: Thread-2 jadlem 100 razy
Filozof: Thread-0 jadlem 200 razy
Filozof: Thread-2 jadlem 200 razy
Filozof: Thread-0 jadlem 300 razy
Filozof: Thread-2 jadlem 300 razy
Filozof: Thread-4 jadlem 100 razy
Filozof: Thread-4 jadlem 200 razy
Filozof: Thread-2 jadlem 400 razy
Filozof: Thread-2 jadlem 500 razy
Filozof: Thread-4 jadlem 300 razy
Filozof: Thread-1 jadlem 100 razy

```

3.2 Implementacja rozwiązania z jednoczesnym podnoszeniem widelców

Implementację drugiego sposobu podnoszenia widelców przedstawia kod (3). Zmianie uległa klasa **Filozof**, która podnosi jeden widelec metodą `podnies()`, gdzie jej argumentem jest drugi widelec. Klasa **Widelec** podczas podnoszenia obsługuje oba widelce przy próbie podniesienia chociaż jednego. Takie samo zachowanie realizuje metoda `odloz()`.

```

1  class Widelec {
2      private boolean zajety = false;
3      public synchronized void podnies(Widelec drugi) {
4          while (zajety && drugi.zajety) {
5              try {
6                  wait();
7              } catch (InterruptedException e) {
8                  Thread.currentThread().interrupt();
9              }
10         }
11         zajety = true;
12         drugi.zajety = true;
13     }
14     public synchronized void odloz(Widelec drugi) {
15         synchronized (this) {

```

```

16         zajety = false;
17         notify();
18     }
19     synchronized (drugi) {
20         drugi.zajety = false;
21         drugi.notify();
22     }
23 }
24 }
25 class Filozof extends Thread {
26     private int _licznik = 0;
27     private Widelec lewyWidelec, prawyWidelec;
28     public Filozof(Widelec lewyWidelec, Widelec prawyWidelec) {
29         this.lewyWidelec = lewyWidelec;
30         this.prawyWidelec = prawyWidelec;
31     }
32     public void run() {
33         while (true) {
34             synchronized (lewyWidelec) {
35                 lewyWidelec.podnies(prawyWidelec);
36                 ++_licznik;
37                 try {
38                     Thread.sleep((int)(50*Math.random()));
39                 } catch (InterruptedException e) {
40                     throw new RuntimeException(e);
41                 }
42                 if (_licznik % 100 == 0) {
43                     System.out.println("Filozof: " + Thread.currentThread().getName() +
44                         " jadłem " + _licznik + " razy");
45                 }
46                 lewyWidelec.odloz(prawyWidelec);
47             }
48         }
49     }
50 }
51 public class Fil5mon {
52     public static void main(String[] args) {
53         int liczbaFilozofow = 5;
54         Widelec[] widelce = new Widelec[liczbaFilozofow];
55         Filozof[] filozofowie = new Filozof[liczbaFilozofow];
56         for (int i = 0; i < liczbaFilozofow; i++) {
57             widelce[i] = new Widelec();
58         }
59         for (int i = 0; i < liczbaFilozofow; i++) {
60             Widelec lewyWidelec = widelce[i];
61             Widelec prawyWidelec = widelce[(i + 1) % liczbaFilozofow];
62             filozofowie[i] = new Filozof(lewyWidelec, prawyWidelec);
63             filozofowie[i].start();
64         }
65     }
66 }

```

Kod Java 3: Implementacja jednoczesnego podnoszenia widelców

W tym przypadku będziemy mieć do czynienia z problemem zakleszczenia. Który występuje dość szybko. Poniższy log z wykonania programu pokazuje tę sytuację:

```

Filozof: Thread-1 jadłem 100 razy
Filozof: Thread-0 jadłem 100 razy
Filozof: Thread-2 jadłem 100 razy
Filozof: Thread-3 jadłem 100 razy

```

Zakleszczenie wynika z tego, że kiedy filozof próbuje jednocześnie podnieść widelec, może zdarzyć się,

że nie będzie w stanie ponieść drugiego ponieważ on jest zajęty. W takim przypadku dochodzi do zablokowania systemu.

3.3 Implementacja rozwiązania z lokajem

Kod (4) implementację problemu pięciu filozofów, wykorzystującą mechanizm lokaja. Kod którego, użyłem to zmodyfikowana wersja (2). Każdy obiekt klasy **Filozof** posiada pole lokaj. Przed próbą podniesienia widelca każdy filozof, najpierw wywołuje na lokaju metodę przedJedzeniem(), a po odłożeniu widelców poJedzeniu(). Klasa **Lokaj** posiada dwa pola, *liczba filozofów* - 1 oraz *liczbę aktualnie jedzących*. Posiada dwie metody. Pierwsza przedJedzeniem sprawdza czy nie ma więcej jedzących niż może być ewentualnie czeka lub dodaje nowego jedzącego. Druga poJedzeniu zmniejsza aktualną liczbę jedzących.

```
1 class Lokaj {
2     private final int limit;
3     private int jedzacy = 0;
4
5     public Lokaj(int limit) {
6         this.limit = limit;
7     }
8     public synchronized void przedJedzeniem() throws InterruptedException {
9         while (jedzacy >= limit) {
10             wait();
11         }
12         jedzacy++;
13     }
14     public synchronized void poJedzeniu() {
15         jedzacy--;
16         notifyAll();
17     }
18 }
19 class Widelec {
20     private boolean zajety = false;
21     public synchronized void podnies() {
22         while (zajety) {
23             try {
24                 wait();
25             } catch (InterruptedException e) {
26                 Thread.currentThread().interrupt();
27             }
28         }
29         zajety = true;
30     }
31     public synchronized void odloz() {
32         zajety = false;
33         notify();
34     }
35 }
36 class Filozof extends Thread {
37     private final Widelec lewyWidelec;
38     private final Widelec prawyWidelec;
39     private final Lokaj lokaj;
40     private int _licznik = 0;
41
42     public Filozof(Widelec lewyWidelec, Widelec prawyWidelec, Lokaj lokaj) {
43         this.lewyWidelec = lewyWidelec;
44         this.prawyWidelec = prawyWidelec;
45         this.lokaj = lokaj;
46     }
47     public void run() {
48         while (true) {
```

```

49         try {
50             lokaj.przedJedzeniem();
51             lewyWidelec.podnies();
52             prawyWidelec.podnies();
53             ++_licznik;
54             try {
55                 Thread.sleep((int)(50*Math.random()));
56             } catch (InterruptedException e) {
57                 throw new RuntimeException(e);
58             }
59             if (_licznik % 100 == 0) {
60                 System.out.println("Filozof: " + Thread.currentThread().getName() +
61                     " jadlem " + _licznik + " razy");
62             }
63
64             prawyWidelec.odloz();
65             lewyWidelec.odloz();
66             lokaj.poJedzeniu();
67         } catch (InterruptedException e) {
68             Thread.currentThread().interrupt();
69         }
70     }
71 }
72 }
73 public class Fil5mon {
74     public static void main(String[] args) {
75         int liczbaFilozofow = 5;
76         Widelec[] widelce = new Widelec[liczbaFilozofow];
77         Filozof[] filozofowie = new Filozof[liczbaFilozofow];
78         Lokaj lokaj = new Lokaj(liczbaFilozofow - 1);
79         for (int i = 0; i < liczbaFilozofow; i++) {
80             widelce[i] = new Widelec();
81         }
82         for (int i = 0; i < liczbaFilozofow; i++) {
83             Widelec lewyWidelec = widelce[i];
84             Widelec prawyWidelec = widelce[(i + 1) % liczbaFilozofow];
85             filozofowie[i] = new Filozof(lewyWidelec, prawyWidelec, lokaj);
86             filozofowie[i].start();
87         }
88     }
89 }

```

Kod Java 4: Implementacja wykorzystująca klasę **Lokaj**

Zastosowanie mechanizmu Lokaja jako dodatkowej synchronizacji spowodowało wyeliminowanie możliwości zakleszczenia oraz zagłodzenia. Fragment loga z konsoli wykonania programu:

```

...
Filozof: Thread-1 jadlem 100 razy
Filozof: Thread-0 jadlem 100 razy
Filozof: Thread-1 jadlem 200 razy
Filozof: Thread-1 jadlem 300 razy
Filozof: Thread-4 jadlem 100 razy
Filozof: Thread-4 jadlem 200 razy
Filozof: Thread-2 jadlem 100 razy
Filozof: Thread-4 jadlem 300 razy
Filozof: Thread-1 jadlem 400 razy
Filozof: Thread-1 jadlem 500 razy
Filozof: Thread-0 jadlem 200 razy
Filozof: Thread-3 jadlem 100 razy
Filozof: Thread-3 jadlem 200 razy

```

Filozof: Thread-2 jadłem 200 razy
Filozof: Thread-2 jadłem 300 razy
Filozof: Thread-2 jadłem 400 razy
Filozof: Thread-2 jadłem 500 razy
Filozof: Thread-4 jadłem 400 razy
Filozof: Thread-3 jadłem 300 razy
Filozof: Thread-1 jadłem 600 razy
Filozof: Thread-0 jadłem 300 razy
...

4 Wnioski dotyczące wydajności

Wykonanie pomiarów czasów w zależności od sposobu realizacji w tym przypadku jest zbędne, ponieważ w dwóch pierwszych implementacjach dochodzi do zakleszczenia. Możemy jednak opisać obserwacje wielokrotnych wykonania poszczególnych wersji.

W trywialnym rozwiązaniu każdy filozof próbuje najpierw podnieść lewy widelec, co często prowadzi do zakleszczenia. System kończy działanie w stanie zakleszczenia, co czyniło to podejście bardzo niewydajnym i niestabilnym.

Rozwiązanie z jednoczesnym podnoszeniem widelców wprowadziło rywalizację o zasoby i zagłodzenie. Filozofowie czasami nie mogli uzyskać obu widelców, co prowadziło do opóźnień. Mimo problemów z wydajnością, niemal każdy filozof mógł zjeść przynajmniej raz, co czyniło to rozwiązanie bardziej sprawiedliwym niż trywialne.

Wykorzystanie lokaja, który kontroluje dostęp do widelców, zapobiegało zakleszczeniu i zagłodzeniu, co zapewniało stabilność i wysoką przepustowość. Lokaj dbał o sprawiedliwy dostęp do widelców, minimalizując rywalizację i poprawiając wydajność.

Czynniki wpływające na wydajność to przede wszystkim:

1. Unikanie zakleszczeń
2. Rywalizacja o zasoby
3. Sprawiedliwość (równomierna liczba zjedzonych porcji przez wszystkich filozofów)

Tylko implementacja z lokajem zapewnia znakomitą wydajność bazując na powyższych czynnikach. Innymi sposobami na zapewnienie sprawiedliwego dostępu do widelców może być dynamiczny czas oczekiwania. Filozof, który długo nie jadł może otrzymać priorytet w dostępie do widelców.

Literatura

[1] <https://home.agh.edu.pl/~balis/dydakt/tw/lab8/tw-5fil.pdf>

[2] https://pl.wikipedia.org/wiki/Problem_ucztuj%C4%85cych_filozof%C3%B3w